VOL. 46

OpenAI API

Build applications using OpenAI's models (GPT, DALL-E, Embeddings, Whisper)

Source	platform.openai.com/docs
Generated	2026-10-11
Level	Intermediate
Estimated study time	25 hours
Language	English
Volume	Vol. 46 - OpenAI API

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 4

Chapter 1 - Getting started with the API 4

1.1 Summary 4

1.2 Concepts 4

1.3 Examples 4

Example 1 - API setup 4

Example 2 - Understanding usage 5

1.4 Mini test 5

Chapter 2 - Chat models 6

2.1 Summary 6

2.2 Concepts 6

2.3 Examples 6

Example 1 - Multi-turn conversation 6

Example 2 - Streaming responses 7

2.4 Mini test 7

Chapter 3 - Embeddings 8

3.1 Summary 8

3.2 Concepts 8

3.3 Examples 8

Example 1 - Creating and comparing embeddings 8

3.4 Mini test 9

Chapter 4 - Image generation 9

4.1 Summary 9

4.2 Concepts 10

4.3 Examples 10

Example 1 - DALL-E 3 integration 10

4.4 Mini test 11

Chapter 5 - Audio models (Whisper) 11

5.1 Summary 11

5.2 Concepts 11

5.3 Examples 11

Example 1 - Transcription 11

5.4 Mini test . . . 11
Chapter 6 - Error handling and best practices . . . 12
6.1 Summary . . . 12
6.2 Concepts . . . 12
6.3 Examples . . . 12
Example 1 - Robust API client . . . 12
Example 2 - Usage tracking . . . 13
6.4 Mini test . . . 14
Chapter 7 - Where to go next . . . 14
7.1 Summary . . . 14
7.2 Cheat sheet . . . 15
Final test . . . 15
8.1 Final test answers . . . 15
Sources . . . 16

How to use this volume

Prerequisites. Basic Python, familiarity with HTTP requests and API concepts.

Estimated study time. About 25 hours.

About the source. The syllabus follows OpenAI's official API documentation. All explanations, examples and exercises are original. Code examples use the `openai` Python package with a free-tier account.

Chapter 1 - Getting started with the API

1.1 Summary

- The OpenAI API key authenticates your requests.
- Each request is an HTTPS POST to an API endpoint.
- Responses are returned as JSON.
- You pay per token: roughly $0.50 per million tokens for GPT-4o mini.
- Free tier credits are available at platform.openai.com.
- Rate limits vary by model and tier.

1.2 Concepts

Component	Description
API key	Secret string identifying your account
Endpoint	URL for a specific API (e.g., /chat/completions)
Token	Unit of text (~4 characters); you pay per token
Request	JSON body with model, messages, parameters
Response	JSON with model output and metadata
Rate limit	Max requests per minute

1.3 Examples

Example 1 - API setup

Python

```python
# ch01_setup.py
import os
from openai import OpenAI

# Set your API key as an environment variable
# os.environ["OPENAI_API_KEY"] = "sk-..."

# In production, load from environment:
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# A simple function to call the chat API
def chat(prompt, model="gpt-4o-mini", temperature=0.7, max_tokens=150):
    response = client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user", "content": prompt}
        ],
        temperature=temperature,
        max_tokens=max_tokens
    )
    return response.choices[0].message.content, response.usage

# Demo output (not calling the API here)
print("client.chat.completions.create(...)")
print(f"  model: gpt-4o-mini")
print(f"  messages: [system, user]")
print(f"  temperature: 0.7")
print(f"  max_tokens: 150")
```

Example 2 - Understanding usage

Python

```python
# ch01_usage.py
print("API usage structure:")
print("""
Response includes:
  - choices[0].message.content: the text output
  - usage: token counts
  - id, created, model, object
""")

print("\nToken counting example:")
sample = "The quick brown fox jumps over the lazy dog."
word_count = len(sample.split())
char_count = len(sample)
estimated_tokens = char_count / 4

print(f"  Text: '{sample}'")
print(f"  Words: {word_count}")
print(f"  Characters: {char_count}")
print(f"  Estimated tokens: {estimated_tokens:.1f}")
print(f"  Typical range: {int(estimated_tokens - 1)}-{int(estimated_tokens + 1)} tokens")
```

1.4 Mini test

1. How do you authenticate with the OpenAI API? 2. What is a token? 3. What affects API cost?

Answers: 1) With an API key in the Authorization header. 2) A unit of text that the model processes. 3) Number of tokens in input and output.

Chapter 2 - Chat models

2.1 Summary

- Chat models use a messages array with roles: system, user, assistant.
- The system message sets behaviour and constraints.
- Conversation history helps maintain context.
- Parameters: model, messages, temperature, max_tokens, top_p, n.
- Streaming: receive tokens as they are generated.

2.2 Concepts

Parameter	Default	Range	Effect
temperature	1.0	0-2	Randomness
max_tokens	variable	1-4096	Max output length
top_p	1.0	0-1	Nucleus sampling
n	1	1-N	Number of responses
stream	false	bool	Stream tokens

2.3 Examples

Example 1 - Multi-turn conversation

Python

```python
# ch02_conversation.py
# Simulating a multi-turn conversation structure
conversation = [
    {"role": "system", "content": "You are a helpful coding tutor. Explain concepts clearly."},
    {"role": "user", "content": "What is recursion?"},
]

# Assistant response (simulated)
conversation.append({
    "role": "assistant",
    "content": "Recursion is when a function calls itself. It needs a base case to stop.\n\nExample: fa
})

# User's follow-up
conversation.append({
    "role": "user",
    "content": "Can you show the code for factorial?"
})

# To get the next response:
print("Messages sent to API:")
for msg in conversation:
    print(f"  [{msg['role']}]: {msg['content'][:50]}...")

print(f"\nTotal messages: {len(conversation)}")
print(f"Estimated tokens: ~{sum(len(m['content']) for m in conversation) // 4}")
```

Example 2 - Streaming responses

Python

```python
# ch02_streaming.py
print("Streaming API call structure:")
print("""
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[...],
    stream=True  # Important: enables streaming
)

for chunk in response:
    content = chunk.choices[0].delta.content
    if content:
        print(content, end="", flush=True)
""")

print("\nChunks arrive as they are generated:")
chunks = ["Hello", " world", "!", " How", " can", " I", " help", " you", " today?"]
for chunk in chunks:
    print(f"  chunk: '{chunk}'")
```

2.4 Mini test

1. What is the purpose of the system message? 2. How does streaming help? 3. What does top_p=0.1 do?

Answers: 1) Sets the behaviour and constraints of the assistant. 2) Shows output as it's generated. 3) Samples from 10% of likely next tokens, reducing diversity.

Chapter 3 - Embeddings

3.1 Summary

- Embeddings convert text into numerical vectors.
- Similar texts have similar embeddings (close in vector space).
- The text-embedding-3-small and text-embedding-3-large models are available.
- Use cases: semantic search, recommendation, clustering, classification.
- Cosine similarity measures vector similarity.

3.2 Concepts

Concept	Description			
Embedding	Dense vector of floating-point numbers			
Dimension	Length of the vector (e.g., 1536 for text-embedding-3-small)			
Cosine similarity	cos(θ) = (A·B)/(	A	B	); 1=identical, -1=opposite, 0=orthogonal
Euclidean distance	Straight-line distance in vector space			

3.3 Examples

Example 1 - Creating and comparing embeddings

Python

```python
# ch03_embeddings.py
import numpy as np

# Simulated embeddings (in practice, use client.embeddings.create)
np.random.seed(42)

# Three sentences about similar and different topics
# In practice these would be 1536-dimensional vectors
apple_emb = np.array([0.9, 0.1, 0.0, 0.2])   # about fruit/apple
tech_emb = np.array([0.8, 0.2, 0.1, 0.1])    # about technology/apple
car_emb = np.array([0.1, 0.9, 0.8, 0.3])     # about cars

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print("Cosine similarities:")
print(f"  apple vs tech: {cosine_similarity(apple_emb, tech_emb):.3f}")
print(f"  apple vs car: {cosine_similarity(apple_emb, car_emb):.3f}")
print(f"  tech vs car: {cosine_similarity(tech_emb, car_emb):.3f}")

print("\nIn practice:")
print("""
from openai import OpenAI
client = OpenAI()

response = client.embeddings.create(
    model="text-embedding-3-small",
    input=["I love apples", "Apple released a new iPhone", "I need a new car"]
)

for i, emb in enumerate(response.data):
    print(f"Embedding {i}: dimensions={len(emb.embedding)}, first 5: {emb.embedding[:5]}")
""")
```

output

```
Cosine similarities:
  apple vs tech: 0.979
  apple vs car: 0.476
  tech vs car: 0.517
```

3.4 Mini test

1. What do embeddings represent? 2. What is cosine similarity? 3. What are common uses?

Answers: 1) Semantic meaning as numerical vectors. 2) A measure of vector similarity based on angle. 3) Search, clustering, classification, recommendation.

Chapter 4 - Image generation

4.1 Summary

- DALL-E generates images from text prompts.
- The images-generations endpoint returns a URL to the generated image.

- You can specify size (e.g., 1024x1024), quality, and style.
- Images can be modified using image editing endpoints.
- Usage is billed per image (e.g., $0.04 for 1024x1024).

4.2 Concepts

Parameter	Options	Description
n	1-10 (or 1-4 for dall-e-3)	Number of images
size	1024x1024, 1024x1792, 1792x1024	Image dimensions
quality	standard, hd	Quality and cost
style	vivid, natural	Style of generation

4.3 Examples

Example 1 - DALL-E 3 integration

Python

```python
# ch04_dalle.py
print("DALL-E 3 API call structure:")
print("""
response = client.images.generate(
    model="dall-e-3",
    prompt="A watercolor painting of a futuristic city at sunset, "
           "with flying cars and neon lights, detailed, atmospheric",
    n=1,
    size="1024x1024",
    quality="standard",
    style="vivid",
    response_format="url"  # or "b64_json"
)

image_url = response.data[0].url
print(f"Generated image: {image_url}")

# To download and save:
# import urllib.request
# urllib.request.urlretrieve(image_url, "generated.png")
""")

print("\nDALL-E 2 API (for variations):")
print("""
import openai
response = openai.Image.create_variation(
    image=open("original.png", "rb"),
    n=4,
    size="512x512"
)
for i, img in enumerate(response.data):
    print(f"Variation {i}: {img.url}")
""")
```

4.4 Mini test

1. What model does DALL-E 3 use? 2. What does the `style` parameter control? 3. What are the two response formats?

Answers: 1) "dall-e-3". 2) The artistic style (vivid vs natural). 3) "url" or "b64_json".

Chapter 5 - Audio models (Whisper)

5.1 Summary

- Whisper transcribes and translates audio.
- The audio/transcriptions endpoint accepts audio files.
- Supported formats: mp3, wav, mp4, webm, etc.
- Output: text transcription with timestamps.
- The audio/translations endpoint translates to English.

5.2 Concepts

Endpoint	Purpose	Input	Output
/v1/audio/transcriptions	Speech to text	Audio file + language	Text
/v1/audio/translations	Translate to English	Audio file	English text

5.3 Examples

Example 1 - Transcription

Python

```python
# ch05_whisper.py
print("Whisper transcription API:")
print("""
response = client.audio.transcriptions.create(
    file=open("audio.mp3", "rb"),
    model="whisper-1",
    response_format="text",  # text, verbose_json, srt, etc.
    prompt="Optional context: this is a technical lecture",
    language="en"  # optional
)

print(response)  # The transcribed text
""")
```

5.4 Mini test

1. What model is used for audio? 2. What formats are supported? 3. What does response_format do?

Answers: 1) "whisper-1". 2) mp3, wav, mp4, webm, and others. 3) Controls the output structure.

Chapter 6 - Error handling and best practices

6.1 Summary

- Always wrap API calls in try/except.
- Handle rate limits (HTTP 429) with exponential backoff.
- Check your usage and billing.
- Cache responses to reduce cost during development.
- Never expose the API key in client-side code.
- Use environment variables for secrets.

6.2 Concepts

Error	HTTP Status	Meaning
Rate limit exceeded	429	Too many requests
Authentication failed	401	Invalid or missing API key
Insufficient quota	429	Billing limit reached
Invalid request	400	Missing or invalid parameters
Server error	500	OpenAI server issue

6.3 Examples

Example 1 - Robust API client

Python

```python
# ch06_robust.py
import os
import time
from openai import OpenAI, RateLimitError, APIError

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def safe_chat(messages, model="gpt-4o-mini", max_retries=3):
    for attempt in range(max_retries):
        try:
            response = client.chat.completions.create(
                model=model,
                messages=messages,
                temperature=0.7
            )
            return response.choices[0].message.content, response.usage
        except RateLimitError:
            wait_time = 2 ** attempt  # exponential backoff
            print(f"Rate limited. Waiting {wait_time}s...")
            time.sleep(wait_time)
        except APIError as e:
            print(f"API error: {e}")
            if attempt == max_retries - 1:
                raise
            time.sleep(1)
    raise Exception("Max retries exceeded")

# Usage
messages = [
    {"role": "user", "content": "Hello!"}
]
# result, usage = safe_chat(messages)
# print(f"Response: {result}")
# print(f"Tokens used: {usage.total_tokens}")
```

Example 2 - Usage tracking

Python

```python
# ch06_usage.py
print("Usage tracking:")
print("""
from openai import OpenAI
client = OpenAI()

# Check model list
models = client.models.list()
for model in models.data:
    print(model.id)
""")

print("\nCost estimation:")
pricing = {
    "gpt-4o-mini": {"prompt": 0.15, "completion": 0.60},  # $/1M tokens
    "gpt-4o": {"prompt": 3.00, "completion": 15.00},
}

for model, prices in pricing.items():
    print(f"\n{model}:")
    print(f"  Input: ${prices['prompt']}/1M tokens")
    print(f"  Output: ${prices['completion']}/1M tokens")
    print(f"  1000 tokens ≈ ${prices['prompt'] * 1000 / 1000000:.4f} (input)")
```

output

```
Cost estimation:

gpt-4o-mini:
  Input: $0.15/1M tokens
  Output: $0.60/1M tokens
  1000 tokens ≈ $0.000150 (input)

gpt-4o:
  Input: $3.00/1M tokens
  Output: $15.00/1M tokens
  1000 tokens ≈ $0.003000 (input)
```

6.4 Mini test

1. What is exponential backoff? 2. How to handle 429 errors? 3. Why use env variables for API keys?

Answers: 1) Increasing wait times between retries. 2) Wait and retry. 3) To keep secrets out of source code.

Chapter 7 - Where to go next

7.1 Summary

- Read the OpenAI cookbook for examples and patterns.
- Explore the assistants API for building AI assistants.
- Try fine-tuning for custom model behaviour.

- Check the rate limits and usage limits on the dashboard.
- Build a small project: chatbot, summariser, tutor, or agent.

7.2 Cheat sheet

Endpoint	Purpose
/chat/completions	Text chat and generation
/completions	Legacy text completion
/embeddings	Text to vector
/images/generations	DALL-E image generation
/images/edits	Image modification
/audio/transcriptions	Speech to text (Whisper)
/audio/translations	Translate to English
/files	File management
/fine_tuning/jobs	Create fine-tuned models

Final test

1. What is needed to authenticate with the OpenAI API?
2. What are the three message roles in chat?
3. What does temperature=0.2 do?
4. What does max_tokens=200 limit?
5. What are embeddings used for?
6. What format does DALL-E produce images in?
7. What is exponential backoff?
8. What does top_p=0.5 mean?
9. What model does Whisper use?
10. How to handle rate limit errors?

8.1 Final test answers

1. An API key set via OpenAI client or Authorization header.
2. system, user, assistant.
3. Makes output more deterministic and focused.
4. The maximum number of tokens in the response.

5. Semantic search, clustering, classification, similarity.
6. Image URLs (response_format="url") or base64 (response_format="b64_json").
7. Doubling the wait time between each retry attempt.
8. Samples from the top 50% of likely next tokens (nucleus sampling).
9. Model "whisper-1".
10. Catch the error, wait, and retry with exponential backoff.

Sources

- https://platform.openai.com/docs
- https://platform.openai.com/docs/guides/rate-limits
- https://platform.openai.com/docs/guides/sampling
- https://platform.openai.com/docs/guides/audio
- https://platform.openai.com/docs/guides/images